Experiment 1a: Data Preprocessing Using Iris Dataset

Aim

To perform data preprocessing operations on the Iris dataset by encoding categorical variables, feature scaling, and splitting the dataset into training and testing sets.

Procedure

1. Install and import the required Python libraries such as Pandas, NumPy, Seaborn, Matplotlib, and Scikit-learn.
2. Load the Iris dataset using the Scikit-learn library.
3. Convert the dataset into a Pandas DataFrame.
4. Explore the dataset using:
   - `head()`
   - `info()`
   - `describe()`
   - `isnull().sum()`
5. Encode the species labels into numerical values using LabelEncoder.
6. Standardize numerical features using StandardScaler.
7. Split the dataset into training and testing data using `train_test_split()`.
8. Display the preprocessed data and dataset shapes.

Code

```
!pip install pandas numpy matplotlib seaborn scikit-learn

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

df['species'] = df['species'].map({
```

```
    0: 'setosa',
    1: 'versicolor',
    2: 'virginica'
})

print(df.head())

print(df.info())

print(df.describe())

print(df.isnull().sum())

le = LabelEncoder()
df['species'] = le.fit_transform(df['species'])

scaler = StandardScaler()
num_cols = df.columns[:-1]
df[num_cols] = scaler.fit_transform(df[num_cols])

X = df.drop('species', axis=1)
y = df['species']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training Data Shape:", X_train.shape)
print("Test Data Shape:", X_test.shape)

print(X_train.head())
```

Result

The Iris dataset was successfully preprocessed by encoding the species labels, scaling features, and splitting the data into training and testing sets.